

Métodos default

Los métodos default fueron incorporados en Java 8. Fueron creados con un solo propósito, no tener que escribir un método añadido en la interfaz en cada una de las clases que implementa esta interfaz. Vamos a demostrarlo con un ejemplo, imaginad un caso en el que tenemos la **Interfaz Vehículo y dos clases(Coche y tractor)**:

```
interface Vehiculo {  
    void arrancar();  
}
```

DATO IMPORTANTE ⇒ Antes de la incorporación de los métodos default los métodos de las interfaces no poseían cuerpo ni llaves.

```
public class Coche implements Vehiculo{  
  
    @Override  
    public void arrancar() {  
        // TODO Auto-generated method stub  
    }  
  
}
```

En el caso de que quisiéramos implementar por ejemplo un método que se llamase void tocarBocina(); , al agregarlo a la interfaz Vehículo, dejarían de compilar los códigos de las clases Coche y Tractor, además del hecho de que luego de agregar el método tocarBocina(); habría que escribir el cuerpo del método en cada clase. Ahora es sencillo porque solo tenemos dos.Sin embargo, imaginaros que la interfaz estuviese implementada con 1000 clases, un disparate.

El proceso que usaban los programadores antes era el siguiente:

- Creabas la Interfaz con métodos sin cuerpo.
- Creabas una clase abstracta que implementaba la interfaz y escribía el código común.
- Tus clases finales heredaban de esa clase abstracta.

Un proceso que ahora los programadores los vemos y no nos creemos que se hiciese de ese modo hasta el 2014.

```
public class Tractor implements Vehiculo{  
  
    @Override  
    public void arrancar() {  
        // TODO Auto-generated method stub  
  
    }  
}
```

Los métodos default nos permiten:

1. Generalizar un método para todas las clases en las que se implementa la interfaz del método.
2. Crear un cuerpo para el método dentro de la interfaz directamente en ella
3. Un código más limpio y estructurado.
4. Evitar la creación de clases abstractas para luego conectarlas con otras clases, por tanto, nos ahorramos espacio y trabajo.

Métodos private

Los métodos private fueron incorporados en Java 9 y llegaron para poner orden y limpieza a los métodos default. ¿Qué son los métodos private, para que sirven y cuáles son sus características?.

Los métodos private son métodos que como bien dice son métodos PRIVADOS que otorgan a los métodos default una mejor organización y seguridad al código.

Sirven para compartir mejor la lógica interna entre métodos default o static de una interfaz. Además de eso, estos métodos no son visibles fuera de la interfaz, ni se heredan ni pueden ser implementados por las clases que implementan su interfaz.

Vamos a continuar con el mismo ejemplo de antes: **Interfaz Vehículo y dos clases(Coche y tractor):**

```
private void cinturonesAbrochados() {
    System.out.println("Cinturones abrochados correctamente alrededor de los huesos de la cintura");
}

default void encenderMotor() {
    //lo que hacemos aquí es llamar al método private cuya función es este caso sería
    //comprobar si los cinturones están correctamente abrochados para poder arrancar
    //el motor
    cinturonesAbrochados();
    System.out.println("Motor encendido y listo.");
}
```

En este caso utilizamos el método private **private void cinturonesAbrochados()** que funciona cómo un método de seguridad para que el usuario pueda arrancar el motor. No obstante, su función más importante es abreviar el código del método default void encenderMotor() con la finalidad de crear un código más limpio y más adverso a futuras implementaciones a la interfaz.

```
public class Principal {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        Coche c1 = new Coche();

        c1.encenderMotor();

    }
}
```

Ahora al crear un objeto tipo coche en la clase Principal nos deja llamar al método default encenderMotor() que dentro si recordamos lleva el método private cinturonesAbrochados().

```
10     c1.encenderMotor();
11
12 }
```

Console X

<terminated> Principal (12) [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (26 mar 2026, 20:39:50 – 20:39:51 elapsed 0:00:00) [pid: 6

Cinturones abrochados correctamente alrededor de los huesos de la cintura
Motor encendido y listo.

Cómo podemos comprobar, el método private hace su función interna de comprobar los cinturones y se escribe el mensaje de su cuerpo.

EN RESUMEN ➡ Los métodos private sirven para la reutilización del código y para proporcionar un orden al código.

EJEMPLO PRÁCTICO

<https://github.com/gilgumar25-creator/proyectoInterfacesProgramacion.git>